HOBBLY

Transaction State Machine & Services Platform

Deterministic FSM, Idempotent Orchestration & Escrow Economics for Commerce

Igor Grybek
Systems Architect, Arxa Group
Live Case Study | GitHub Repository
Standards: PCI-DSS | PSD2 / SCA

Executive Summary & Strategic Context

In two-sided booking marketplaces and service-commerce platforms, transactions represent extended, asynchronous operational commitments rather than instant retail purchases. Platforms frequently display rapid Gross Merchandise Value (GMV) growth while suffering severe operating cash burn due to hidden transaction friction: inventory race conditions, double charging under network latency, unhandled merchant cancellations, and manual customer support escalations.

Hobbly was engineered at **Arxa Group** to enforce transaction integrity. Hobbly operates under the defining principle: *exceptions determine the operating model*. Utilizing a **Deterministic Finite State Automaton (FSM)**, SHA-256 idempotency request fingerprinting, and automated double-entry escrow reconciliations, Hobbly ensures that concurrent bookings, payment captures, cancellations, and dispute arbitration are resolved deterministically, guaranteeing contribution margin profitability on every completed transaction.

System Architecture & State Machine Mechanics

Hobbly isolates transaction states, payment tokens, and operational workflows into four modular layers:

1. **Atomic Inventory Reservation Core:** Manages listing capacity via transactional mutex locks and automated expiration countdowns. Completely prevents double-booking and inventory overselling during high-concurrency checkout traffic.
2. **Cryptographic Idempotency Filter:** Every client payment request requires a unique `Idempotency-Key`. Repeated network attempts return cached transaction results without triggering duplicate charges. Reusing an idempotency key across differing booking IDs is immediately blocked with a `409 CONFLICT` security fault.
3. **Deterministic Finite State Automaton (FSM):** Governs transitions across explicit states: $\mathcal{S} = \{$PUBLISHED, RESERVED, PEN
Invalid transitions are strictly rejected.
4. **Dual-Entry Escrow Reconciliation Ledger:** Maintains real-time balance integrity across gross captured funds, payment processor fees, merchant escrow, and automated refund disbursements. Guarantees zero unallocated financial drift.

5-Dimensional Multidimensional Operating Model

In accordance with Arxa Group Systems Architecture Standards, Hobbly integrates five operational dimensions:

- **Technical Architecture:** Python finite state engine with SQLite transaction persistence, atomic ACID database transactions, and OpenAPI 3.1 REST contracts.
- **Operating Workflow (RACI):** Activity Organizer (publishes capacity and confirms fulfillment) $\rightarrow$ End Customer (initiates payment and executes self-service cancellations within policy) $\rightarrow$ Operations Specialist (monitors exception queues and arbitrates disputes) $\rightarrow$ Finance Manager (authorizes batch disbursements).
- **Financial Model & Unit Margin Breakeven:** Modeled on a baseline €50.00 booking at a 10% take-rate (€5.00 revenue). Fixed gateway processing (€1.00) and expected fraud loss (€0.50) leave €3.50 in gross margin. To secure profitability, **customer support friction must remain strictly below €3.50 per transaction**. Hobbly's self-service workflows cap support overhead at €1.50, securing a **40.0% net contribution margin (+€2.00/booking)**.
- **Risk & Governance Model:** Zero-footprint PCI-DSS compliance via tokenized gateway handshakes (Stripe/Adyen). Full alignment with European PSD2 Strong Customer Authentication (SCA) directives and consumer cancellation rights.
- **Scaling & Integration:** Redis Redlock distributed locking for high-volume flash sales and automated multi-currency merchant payout routing.

Empirical Verification & Benchmark Performance

The reference implementation is verified via 8 automated regression test suites in the public repository (hobbly-booking-payments on GitHub):

Benchmark Metric	Observed Score	Unmanaged Baseline	Operational Significance
Reconciliation Balance Leakage	**€0.00**	Variable / Drift	Perfect net zero balance across capture and full-refund flows.
Duplicate Charge Protection	**100.0%**	1.5–3.0% Error	Replay requests return cached receipts; zero duplicate gateway debits.
Cross-Booking Collision Block	**100.0%**	Vulnerable	Conflicting keys rejected immediately with explicit exception audit.
Support Breakeven Headroom	**€2.00 / tx**	-€0.50 / tx	Preserves positive margin against unmanaged manual support escalations.
Net Take-Rate Retention	**40.0%**	-10.0% to 15.0%	Protects platform net cash contribution across all completed transactions.

Architecture Decision Records (ADRs) & API Contracts

All architectural trade-offs are formally governed by RFC-style Architecture Decision Records:

- **ADR-0001:** Deterministic Finite State Automaton (FSM) for Booking Exception Ownership.
- **ADR-0002:** Cryptographic SHA-256 Idempotency Keys for Duplicate Payment Elimination.
- **ADR-0003:** Dual-Entry Escrow Reconciliation & Net Contribution Economics Guardrails.
- **OpenAPI 3.1 Contract:** Endpoints for ‘/v1/listings/id/reserve‘, ‘/v1/payments/capture‘, and ‘/v1/bookings/id/cancel‘.

Methodological & Confidentiality Notice: This architectural whitepaper utilizes validated synthetic benchmark datasets and open institutional commerce frameworks (PCI-DSS, PSD2). No confidential corporate banking or private client data from active or past institutional engagements (BNP Paribas CIB, Santander CIB) is contained herein. © 2026 Igor Grybek. All rights reserved.